Programação funcional

Aula 04 - Conceitos básicos (parte 2)*

*Baseado no material disponível em https://haskell.pesquisa.ufabc.edu.br/haskell

Polimorfismo: Função tamanho

```
tamanho [] = 0
tamanho (_:xs) = 1 + tamanho xs
```

Polimorfismo: Função tamanho

```
*Main> l1 = [1, 2, 3, 4, 5]
*Main> l2 = [True, False]
*Main> l3 = ['O', 'l', 'a']
*Main> :t l1
l1 :: Num a => [a]
*Main> :t l2
l2 :: [Bool]
*Main> :t l3
l3 :: [Char]
```

```
*Main> tamanho l1
5
*Main> tamanho l2
2
*Main> tamanho l3
3
*Main> :t tamanho
tamanho :: Num p => [a] -> p
```

Variável de tipo

Polimorfismo

```
*Main> 5 + 3
8
*Main> 5.3 + 3.5
8.8
*Main> :t 5
5 :: Num p => p
*Main> :t 5.3
5.3 :: Fractional p => p
*Main> :t (+)
(+) :: Num a => a -> a -> a
*Main> let x = 5 :: Int
*Main> :t x
x :: Int
*Main> let y = 6.5 :: Float
*Main> :t y
y :: Float
```

Polimorfismo

```
*Main> x + y

<interactive>:20:5: error:
    • Couldn't match expected type ‘Int’ with actual type ‘Float’
    • In the second argument of ‘(+)’, namely ‘y’
      In the expression: x + y
      In an equation for ‘it’: it = x + y
*Main> :t (+)
(+) :: Num a => a -> a -> a
```

Polimorfismo

- Exemplos de funções

```haskell
impar :: Integral a => a -> Bool
impar n = n `mod` 2 == 1
```

```
*Main> 3 + 5
8
*Main> (+) 3 5
8
*Main> mod 5 2
1
*Main> 5 `mod` 2
1
```

Polimorfismo

```
raizes2Grau :: Floating a => a -> a -> a -> (a, a)
raizes2Grau a b c = (((-b) + sqrt(b^2 - 4 * a * c))/ (2 * a),
                     ((-b) - sqrt(b^2 - 4 * a * c))/ (2 * a))
```

```
*Main> :t raizes2Grau
raizes2Grau :: Floating a => a -> a -> a -> (a, a)
*Main> raizes2Grau 4 3 (-5)
(0.8042476415070754,-1.5542476415070754)
*Main> raizes2Grau 4 3 5
(NaN,NaN)
```

Polimorfismo: cláusula where

```haskell
raizes2GrauV2 :: Floating a => a -> a -> a -> (a, a)
raizes2GrauV2 a b c = (x1, x2)
  where
    x1 = ((-b) + sqDelta) / (2 * a)
    x2 = ((-b) - sqDelta) / (2 * a)
    sqDelta = sqrt delta
    delta = b^2 - 4 * a * c
```

```
*Main> raizes2GrauV2 4 3 5
(NaN,NaN)
*Main> raizes2GrauV2 4 3 (-5)
(0.8042476415070754,-1.5542476415070754)
```

Condicionais

Condicionais

Restringe o tipo a a ser um tipo numérico e possível de ser ordenado

```
abs :: (Ord a, Num a) => a -> a
abs n = if (n >= 0) then n else (-n)
```

```
abs :: (Ord a, Num a) => a -> a
abs n = if (n >= 0)
        then n
        else (-n)
```

Condicionais

- Encadeando

```haskell
signum :: (Ord a, Num a) => a -> a
signum n = if (n == 0)
           then 0
           else if (n > 0)
                then 1
                else (-1)
```

Condicionais

- Reescrevendo a função de raiz

```haskell
raiz2Grau :: (Ord a, Floating a) => a -> a -> a -> (a, a)
raiz2Grau a b c = (x1, x2)
  where
    x1 = if delta >= 0
         then ((-b) + sqDelta) / (2*a)
         else 0
    x2 = if delta >= 0
         then ((-b) - sqDelta) / (2*a)
         else 0
    sqDelta = sqrt delta
    delta = b^2 - 4*a*c
```

Condicionais

- Reescrevendo a função de raiz

```haskell
raiz2Grau :: (Ord a, Floating a) => a -> a -> a -> (a, a)
raiz2Grau a b c = (x1, x2)
  where
    x1 = if delta >= 0
         then ((-b) + sqDelta) / (2*a)
         else 0
    x2 = if delta >= 0
         then ((-b) - sqDelta) / (2*a)
         else 0
    sqDelta = sqrt delta
    delta = b^2 - 4*a*c
```

```haskell
raiz2Grau 4 3 (-5)
raiz2Grau 4 3 5
```

Condicionais

- Reescrevendo a função de raiz

```haskell
raiz2Grau :: (Ord a, Floating a) => a -> a -> a -> (a, a)
raiz2Grau a b c = (x1, x2)
  where
    x1 = if delta >= 0
         then ((-b) + sqDelta) / (2*a)
         else 0
    x2 = if delta >= 0
         then ((-b) - sqDelta) / (2*a)
         else 0
    sqDelta = sqrt delta
    delta = b^2 - 4*a*c
```

```haskell
raiz2Grau 4 3 (-5)
raiz2Grau 4 3 5
```

```
(0.8042476415070754,-1.5542476415070754)
(0.0,0.0)
```

Expressões guardadas (*Guard Expressions*)

- Lido como *tal que*

```
signum :: (Ord a, Num a) => a -> a
signum n | n == 0     =  0  -- signum n tal que n==0
                            -- é definido como 0
         | n > 0      =  1
         | otherwise = -1
```

Expressões guardadas (*Guard Expressions*)

```haskell
classificaIMC :: Double -> String
classificaIMC imc
    | imc <= 18.5 = "abaixo do peso"
    -- não preciso fazer && imc > 18.5
    | imc <= 25.0 = "no peso correto"
    | imc <= 30.0 = "acima do peso"
    | otherwise   = "muito acima do peso"
```

Expressões guardadas (*Guard Expressions*)

- Reescrevendo raiz quadrada
  - O uso de da função **error** interrompe a execução do programa

```
raiz2Grau :: (Ord a, Floating a) => a -> a -> a -> (a, a)
raiz2Grau a b c
  | delta >= 0    = (x1, x2)
  | otherwise     = error "Delta negativo."
  where
    x1      = ((-b) + sqDelta) / (2*a)
    x2      = ((-b) - sqDelta) / (2*a)
    sqDelta = sqrt delta
    delta   = b^2 - 4*a*c
```

Expressões guardadas (*Guard Expressions*)

- Reescrevendo raiz quadrada
  - O uso de da função **error** interrompe a execução do programa

```
raiz2Grau :: (Ord a, Floating a) => a -> a -> a -> (a, a)
raiz2Grau a b c
  | delta >= 0    = (x1, x2)
  | otherwise     = error "Delta negativo."
  where
    x1      = ((-b) + sqDelta) / (2*a)
    x2      = ((-b) - sqDelta) / (2*a)
    sqDelta = sqrt delta
    delta   = b^2 - 4*a*c
```

```
raiz2Grau 4 3 (-5)
raiz2Grau 4 3 5
```

Expressões guardadas (*Guard Expressions*)

- Reescrevendo raiz quadrada
  - O uso de da função **error** interrompe a execução do programa

```
raiz2Grau :: (Ord a, Floating a) => a -> a -> a -> (a, a)
raiz2Grau a b c
  | delta >= 0    = (x1, x2)
  | otherwise     = error "Delta negativo."
  where
    x1      = ((-b) + sqDelta) / (2*a)
    x2      = ((-b) - sqDelta) / (2*a)
    sqDelta = sqrt delta
    delta   = b^2 - 4*a*c
```

```
raiz2Grau 4 3 (-5)
raiz2Grau 4 3 5
```

```
(0.8042476415070754,-1.5542476415070754)
*** Exception: Delta negativo.
```

Pattern matching

```haskell
nao :: Bool -> Bool
nao x = if x then False else True

naoV2 x
  | x = False
  | otherwise = True

naoV3 True = False
naoV3 False = True
```

Pattern matching

- Não precisamos enumerar todos os casos, podemos definir apenas casos especiais

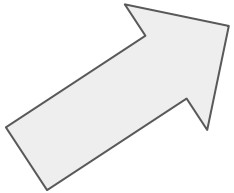
```
soma :: (Eq a, Num a) => a -> a -> a
soma x 0 = x
soma 0 y = y
soma x y = x + y
```

Supondo que (+) seja uma operação muito custosa

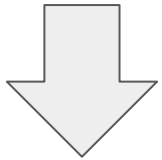
- Assim como os *guards*, os padrões são avaliados do primeiro definido até o último

Pattern matching

```haskell
mul :: Num a => a-> a -> a
mul x y = x*y
```



```haskell
mul :: (Eq a, Num a) => a-> a -> a
mul 0 y = 0
mul x 0 = 0
mul x 1 = x
mul 1 y = y
mul x y = x*y
```



```haskell
mul :: (Eq a, Num a) => a-> a -> a
mul 0 _ = 0
mul _ 0 = 0
mul x 1 = x
mul 1 y = y
mul x y = x*y
```

Expressões λ

- Ou funções anônimas
  -
```
-- Recebe um valor numérico e retorna uma função que
-- recebe um número e retorna outro número
somaMultX :: Num a => a -> (a -> a)
somaMultX x = \y -> x + x * y
-- somaMult2 é uma função que
-- retorna um valor multiplicado por 2
somaMult2 = somaMultX 2
```

Operadores

- Definição de operadores
  - Forma infixa ou forma de função

```
(:+) :: Num a => a -> a -> a
x :+ y = abs x + y
```

```
(:+) :: Num a => a -> a -> a
(:+) x y = abs x + y
```

Listas

Listas

- Uma das principais estruturas em linguagens funcionais.
- Coleção de valores de um determinado tipo.
  - Todos os valores devem ser do mesmo tipo.
- **(:)** - operador de concatenação de elemento com lista

```
*Main> 1 : []
[1]
*Main> 1 : 2 : 3 : []
[1,2,3]
*Main> [1, 2, 3]
[1,2,3]
*Main> :t (:)
(:) :: a -> [a] -> [a]
*Main> 0 : [1, 2, 3]
[0,1,2,3]
```

Listas

- Operador (:)

```
*Main> 0 : 1

<interactive>:6:1: error:
    • Non type-variable argument in the constraint: Num [a]
      (Use FlexibleContexts to permit this)
    • When checking the inferred type
        it :: forall a. (Num a, Num [a]) => [a]
```

Listas

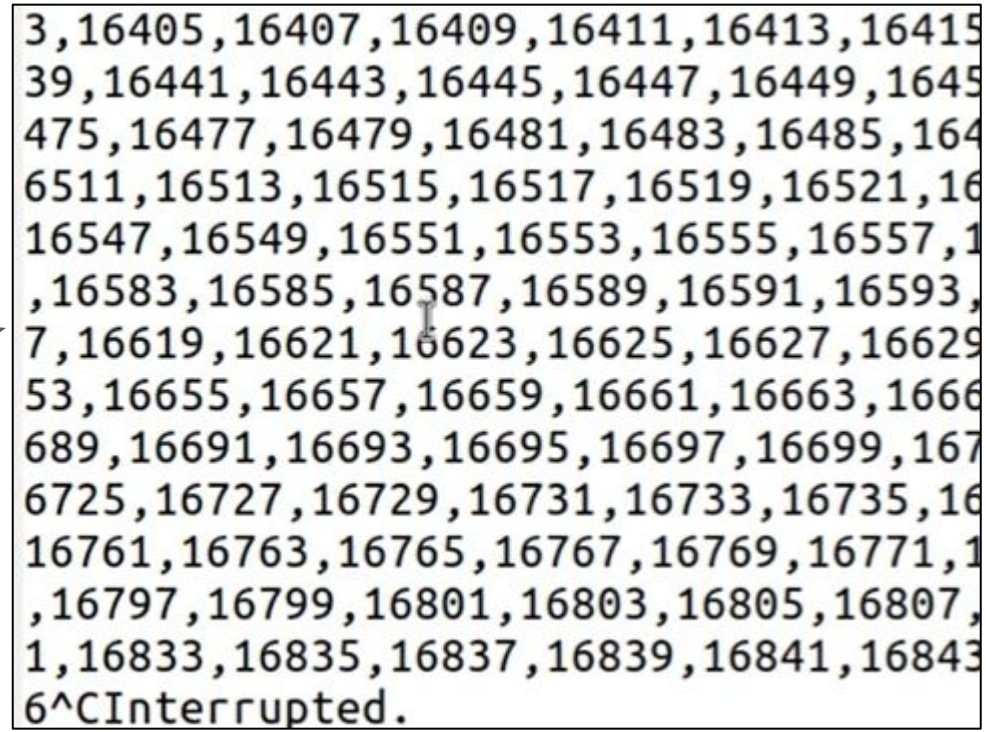
- Definição

```
*Main> [1, 2, 3, 5]
[1,2,3,5]
*Main> ['a', 'e', 'e']
"aee"
*Main> "ola!"
"ola!"
```

```
*Main> "ola!" == ['o', 'l', 'a', '!']
True
```

Lista infinita

```
[1,2,3,4,5,6,7,8,9,10]
*Main> [1,3..10]
[1,3,5,7,9]
*Main> [1,3..]
```



Lista infinita

- Promessa de lista:

```
lista = [0,2..]
```

- Efetivamente:

```
lista = 0 : 2 : geraProximo
```

- Conforme necessário, gera os elementos da lista sequencialmente.
- Então a lista infinita não existe em memória,
  - apenas uma função que gera quantos elementos você precisar dela.

Listas: funções

- Operador !!
  - recupera o i-ésimo elemento da lista
  - índice começando do 0

```
lista = [0..10]
lista !! 2
```

Listas: funções

- Função **head**
  - retorna o primeiro elemento da lista

```
head [0..10]
```

Listas: funções

- Função **tail**
  - retorna a lista sem o primeiro elemento (sua cauda)

```
tail [0..10]
```

Listas: funções

- Exercício
  - O que a expressão abaixo retorna?

```
head (tail [0..10])
```

Listas: funções

- Função **take**
  - retorna os n primeiros elementos da lista:

```
take 3 [0..10]
```

Listas: funções

- Função **drop**
  - retorna a lista sem os n primeiros elementos:

```
drop 6 [0..10]
```

Listas: funções

- Exercício
  - Implemente o operador !! utilizando as funções anteriores.

Listas: funções

- Exercício
  - Implemente o operador !! utilizando as funções anteriores.

```
xs !! n = head (drop n xs)
```

Listas: funções

- Função **length**
  - retorna o tamanho da lista:

```
length [1..10]
```

Listas: funções

- Somatória e Produtória
  - As funções **soma** e **product** retornam a somatória e produtória de uma lista:

```
sum [1..10]
product [1..10]
```

Listas: funções

- Concatenando listas
  - operador ++ concatena duas listas

```
[1..3] ++ [4..10] == [1..10]
```

Listas: funções

- Exercício
  - Implemente a função fatorial utilizando o que aprendemos até agora.

Listas: funções

- Exercício
  - Implemente a função fatorial utilizando o que aprendemos até agora.

```
fatorial n = product [1..n]
```

Listas: *Pattern matching*

- Quais padrões podemos capturar em uma lista?
  - Lista vazia:
    - []
  - Lista com um elemento:
    - (x : []) ou [x]
  - Lista com um elemento seguido de vários outros:
    - (x : xs)

Listas: *Pattern matching*

- Implementando a função **nulo**

```
null :: [a] -> Bool
null [] = True
null _  = False
```

Listas: *Pattern matching*

- Implementando a função **tamanho**

```
length :: [a] -> Int
length [] = 0
length (_:xs) = 1 + length xs
```

Listas: *Pattern matching*

- Implemente a função take. Se n <= 0 deve retornar uma lista vazia.

Listas: *Pattern matching*

- Implemente a função take. Se n <= 0 deve retornar uma lista vazia.

```haskell
take :: Int -> [a] -> [a]
take n _  | n <= 0 = []
take _ []          = []
take n (x:xs)      = x : take (n-1) xs
```

Compreensão de listas

- Na matemática: $\{x^2 \mid x \in \{1..5\}\}$
  - x ao quadrado para todo x do conjunto de 1 a 5
- No Haskell: `[x^2 | x <- [1..5]]`
  - Saída ?

Compreensão de listas

- Na matemática: $\{x^2 \mid x \in \{1..5\}\}$
  - x ao quadrado para todo x do conjunto de 1 a 5
- No Haskell: `[x^2 | x <- [1..5]]`
  - Saída: `[1,4,9,16,25]`

Compreensão de listas

- Na matemática: $\{x^2 \mid x \in \{1..5\}\}$
  - x ao quadrado para todo x do conjunto de 1 a 5
- No Haskell: `[x^2 | x <- [1..5]]`
  - Saída: `[1,4,9,16,25]`
  - x ao quadrado tal que x vem da lista de valores de um a cinco
  - `x <- [1..5]` : expressão geradora

Compreensão de listas

- Exemplos:
  - `[toLower c | c <- "OLA MUNDO"]`
    - Saída: `?`
  - `[(x, even x) | x <- [1,2,3]]`
    - Saída: `?`
  - `[(x,y) | x <- [1..4], y <- [4..5]]`
    - Saída: `?`
  - `[(x,y) | y <- [4..5], x <- [1..4]]`
    - Saída: `?`

Compreensão de listas

- Exemplos:
  - `[toLower c | c <- "OLA MUNDO"]`
    - Saída: `ola mundo`
  - `[(x, even x) | x <- [1,2,3]]`
    - Saída: `?`
  - `[(x,y) | x <- [1..4], y <- [4..5]]`
    - Saída: `?`
  - `[(x,y) | y <- [4..5], x <- [1..4]]`
    - Saída: `?`

Compreensão de listas

- Exemplos:
  - `[toLower c | c <- "OLA MUNDO"]`
    - Saída: `ola mundo`
  - `[(x, even x) | x <- [1,2,3]]`
    - Saída: `[(1,False),(2,True),(3,False)]`
  - `[(x,y) | x <- [1..4], y <- [4..5]]`
    - Saída: `?`
  - `[(x,y) | y <- [4..5], x <- [1..4]]`
    - Saída: `?`

Compreensão de listas

- Exemplos:
  - `[toLower c | c <- "OLA MUNDO"]`
    - Saída: `ola mundo`
  - `[(x, even x) | x <- [1,2,3]]`
    - Saída: `[(1,False),(2,True),(3,False)]`
  - `[(x,y) | x <- [1..4], y <- [4..5]]`
    - Saída:
      `[(1,4),(1,5),(2,4),(2,5),(3,4),(3,5),(4,4),(4,5)]`
  - `[(x,y) | y <- [4..5], x <- [1..4]]`
    - Saída: `?`

Compreensão de listas

- Exemplos:
  - `[toLower c | c <- "OLA MUNDO"]`
    - Saída: `ola mundo`
  - `[(x, even x) | x <- [1,2,3]]`
    - Saída: `[(1,False),(2,True),(3,False)]`
  - `[(x,y) | x <- [1..4], y <- [4..5]]`
    - Saída:
      `[(1,4),(1,5),(2,4),(2,5),(3,4),(3,5),(4,4),(4,5)]`
  - `[(x,y) | y <- [4..5], x <- [1..4]]`
    - Saída:
      `[(1,4),(2,4),(3,4),(4,4),(1,5),(2,5),(3,5),(4,5)]`

Compreensão de listas

- Exemplos:
  - `[(x,y) | x <- [1..4], y <- [4..5]]`
    - Saída:
      `[(1,4),(1,5),(2,4),(2,5),(3,4),(3,5),(4,4),(4,5)]`
  - `[(x,y) | y <- [4..5], x <- [1..4]]`
    - Saída:
      `[(1,4),(2,4),(3,4),(4,4),(1,5),(2,5),(3,5),(4,5)]`
  - Equivalente ao **for** encadeado

Compreensão de listas

- Um gerador pode depender do valor gerado pelo gerador anterior:
  - `[(i,j) | i <- [1..5], j <- [i+1..5]]`
  - Saída:

Compreensão de listas

- Um gerador pode depender do valor gerado pelo gerador anterior:
  - `[(i,j) | i <- [1..5], j <- [i+1..5]]`
  - Saída:
    `[(1,2),(1,3),(1,4),(1,5),(2,3),(2,4),(2,5),(3,4),(3,5),(4,5)]`

Compreensão de listas

- Um gerador pode depender do valor gerado pelo gerador anterior:
  - `[(i,j) | i <- [1..5], j <- [i+1..5]]`
  - Saída: `[(1,2),(1,3),(1,4),(1,5),(2,3),(2,4),(2,5),(3,4),(3,5),(4,5)]`
  - Equivalente a:

```
for (i=1; i<=5; i++) {
  for (j=i+1; j<=5; j++) {
      // faça algo
  }
}
```

Compreensão de listas

- Função **concat**
  - Transforma uma lista de listas em uma lista única
  - `concat [[1,2],[3,4]]`
    - `[1,2,3,4]`
  - Se fosse definida usando compreensão de listas:
    - `concat xss = [x | xs <- xss, x <- xs]`

Compreensão de listas

- Se a função **length** fosse definida com compreensão de listas:
  - `length xs = soma [1 | _ <- xs]`

Compreensão de listas

- Utilizando guardas (*guards*) como filtro
  - `[x | x <- [1..10], even x]`
  - Saída:

Compreensão de listas

- Utilizando guardas (*guards*) como filtro
  - `[x | x <- [1..10], even x]`
  - Saída: `[2,4,6,8,10]`

Compreensão de listas

- Exemplo:
  - Função que retorna lista de todos divisores de n

Compreensão de listas

- Exemplo:
  - Função que retorna lista de todos divisores de n
    - ```
      divisores :: Int -> [Int]
      divisores n = [x | x <- [1..n], n `mod` x == 0]
      ```
      ```

Compreensão de listas

- Exemplo:
  - Utilizando a função **divisores**, definir uma função que retorna **True** se um número é primo

Compreensão de listas

- Exemplo:
  - Utilizando a função **divisores**, definir uma função que retorna **True** se um número é primo
    - ```
      primo :: Int -> Bool
      primo n = divisores n == [1,n]
      ```
      ```

Compreensão de listas

- Exemplo:
  - Utilizando a função **divisores**, definir uma função que retorna **True** se um número é primo
    - ```
      primo :: Int -> Bool
      primo n = divisores n == [1,n]
      ```
  - A avaliação preguiçosa para quando a primeira comparação retorna **False**
      ```

Compreensão de listas

- Exemplo:
  - Gerar uma lista com todos os primos até n

Compreensão de listas

- Exemplo:
  - Gerar uma lista com todos os primos até n
    - ```
      primos :: Int -> [Int]
      primos n = [x | x <- [1..n], primo x]
      ```
      ```

Compreensão de listas

- Exemplo:
  - Lista com todos os primos
    - `todosOsPrimos = [x | x <- [1..], primo x]`

Compreensão de listas

- Exemplo:
  - Lista com todos os primos
    - `todosOsPrimos = [x | x <- [1..], primo x]`
  - n-ésimo primo?

Compreensão de listas

- Exemplo:
  - Lista com todos os primos
    - `todosOsPrimos = [x | x <- [1..], primo x]`
  - n-ésimo primo
    - `nEsimoPrimo n = todosOsPrimos !! (n - 1)`

Compreensão de listas

- Função **zip**
  - Junta duas listas retornando uma lista de pares (tuplas):
  - `zip [1,2,3] [4,5,6]`
  - `zip [1,2,3] ['a', 'b', 'c']`
  - `zip [1,2,3] ['a', 'b', 'c', 'd']`

Compreensão de listas

- Função **zip**
  - Junta duas listas retornando uma lista de pares (tuplas):
  - `zip [1,2,3] [4,5,6]`
    - `[(1,4),(2,5),(3,6)]`
  - `zip [1,2,3] ['a', 'b', 'c']`
  - `zip [1,2,3] ['a', 'b', 'c', 'd']`

Compreensão de listas

- Função **zip**
  - Junta duas listas retornando uma lista de pares (tuplas):
  - `zip [1,2,3] [4,5,6]`
    - `[(1,4),(2,5),(3,6)]`
  - `zip [1,2,3] ['a', 'b', 'c']`
    - `[(1,'a'),(2,'b'),(3,'c')]`
  - `zip [1,2,3] ['a', 'b', 'c', 'd']`

Compreensão de listas

- Função **zip**
  - Junta duas listas retornando uma lista de pares (tuplas):
  - `zip [1,2,3] [4,5,6]`
    - `[(1,4),(2,5),(3,6)]`
  - `zip [1,2,3] ['a', 'b', 'c']`
    - `[(1,'a'),(2,'b'),(3,'c')]`
  - `zip [1,2,3] ['a', 'b', 'c', 'd']`
    - `[(1,'a'),(2,'b'),(3,'c')]`

Compreensão de listas

- Exemplo
  - Criar uma função que recebe uma lista e retorna uma lista com os pares de elementos adjacentes
    - Ex. `pairs [1,2,3]` retorna `[(1,2),(2,3)]`

Compreensão de listas

- Exemplo
  - Criar uma função que recebe uma lista e retorna uma lista com os pares de elementos adjacentes
    - Ex. `pairs [1,2,3]` retorna `[(1,2),(2,3)]`
  - Solução:
    - ```
      pairs :: [a] -> [(a,a)]
      pairs xs = zip xs (tail xs)
      ```
      ```

Compreensão de listas

- Função **and**
  - Recebe uma lista de **Bool** e retorna **True** se todos forem **True** e **False**, caso contrário
    - ```
      and :: [Bool] -> Bool
      and []         = True
      and (True:xs) = and xs
      and _          = False
      ```
  - Também existe a função **or**
      ```

Compreensão de listas

- Exercício
  - Definir uma função **sorted** que retorna **True** se uma lista está ordenada

Compreensão de listas

- Exercício
  - Definir uma função **sorted** que retorna **True** se uma lista está ordenada
    - ```
      sorted :: Ord a => [a] -> Bool
      sorted xs = and [x <= y | (x, y) <- pairs xs]
      ```
      ```

Recursão

Recursão

- Permite expressar ideias declarativas
  - Contém 1 ou mais casos base (para terminar)
  - Conexão com a matemática: $n! = n.(n-1)!$
    - Caso base: $1! = 0! = 1$

Recursão

- Permite expressar ideias declarativas
  - Contém 1 ou mais casos base (para terminar)
  - Conexão com a matemática: $n! = n.(n-1)!$
    - Caso base: $1! = 0! = 1$
  - Em Haskell:
    - ```
      fatorial :: Integer -> Integer
      fatorial 0 = 1
      fatorial 1 = 1
      fatorial n = n * fatorial (n-1)
      ```
      ```

Recursão

- Permite expressar ideias declarativas
  - Contém 1 ou mais casos base (para terminar)
  - Conexão com a matemática: $n! = n.(n-1)!$
    - Caso base: $1! = 0! = 1$
  - Em Haskell:
    - ```
      fatorial :: Integer -> Integer
      fatorial 0 = 1
      fatorial 1 = 1
      fatorial n = n * fatorial (n-1)
      ```

Casos base sempre primeiro
      ```

Recursão

- Fatorial
  - O Haskell avalia a recursão por substituições
    - Não armazena o estado da chamada recursiva em pilha
    -
```
> fatorial 4
    => 4 * fatorial 3
    => 4 * (3 * fatorial 2)
    => 4 * (3 * (2 * fatorial 1))
    => 4 * (3 * (2 * 1))
    => 4 * (3 * 2)
    => 4 * 6
    => 24
```

Recursão

- Máximo divisor comum

Recursão

- Máximo divisor comum
  - ```
    gcd :: Int -> Int -> Int
    gcd 0 b = b
    gcd a 0 = a
    gcd a b = gcd b (a `rem` b)
    ```
    ```

Recursão

- Máximo divisor comum
  - Pilha de tamanho constante
    - Recursão na cauda

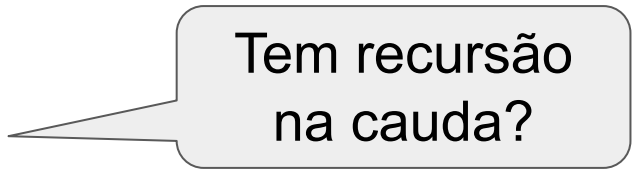
```
> gcd 48 18
    => gcd 18 12
    => gcd 12 6
    => gcd 6 0
    => 6
```

Recursão em listas

- Soma dos elementos de uma lista
  - ```
    soma :: Num a => [a] -> a
    soma []     = 0
    soma (n:ns) = n + soma ns
    ```
    ```

Recursão em listas

- Soma dos elementos de uma lista
  - ```
    soma :: Num a => [a] -> a
    soma []     = 0
    soma (n:ns) = n + soma ns
    ```


    ```

Recursão em listas

- Soma dos elementos de uma lista com recursão na cauda

```haskell
soma :: Num a => [a] -> a
soma [] = 0
soma ns = soma' ns 0
  where
    soma' [] s     = s
    soma' (n:ns) s = soma' ns (n+s)
```

Recursão múltipla

- Quando o retorno da função é a chamada dela mesma múltiplas vezes
- Exemplo: Fibonacci

Recursão múltipla

- Quando o retorno da função é a chamada dela mesma múltiplas vezes
- Exemplo: Fibonacci

```
fib :: Int -> Int
fib 0 = 1
fib 1 = 1
fib n = fib (n-1) + fib (n-2)
```

Recursão: dicas

```
> drop 3 [1..10]
[4,5,6,7,8,9,10]
```

1. Defina a assinatura

```
drop :: Int -> [a] -> [a]
```

2. Enumere os casos

```
drop :: Int -> [a] -> [a]
drop 0 []     =
drop 0 (x:xs) =
drop n []     =
drop n (x:xs) =
```

3. Defina os casos simples

```
drop :: Int -> [a] -> [a]
drop 0 []     = []
drop 0 (x:xs) = x:xs
drop n []     = []
drop n (x:xs) =
```

Recursão: dicas

```
> drop 3 [1..10]
[4,5,6,7,8,9,10]
```

4. Defina os casos restantes

```
drop :: Int -> [a] -> [a]
drop 0 []     = []
drop 0 (x:xs) = x:xs
drop n []     = []
drop n (x:xs) = drop (n-1) xs
```

5. Generalize e simplifique

```
drop :: Int -> [a] -> [a]
drop _ []     = []
drop 0 xs     = xs
drop n (x:xs) = drop (n-1) xs
```

Referências

- Material do prof. Emilio Francesquini
  - https://haskell.pesquisa.ufabc.edu.br/haskell